Working with Claude Code

A practical walkthrough of how I use Claude Code to build real software.

This deck pairs with the **claudecode** starter repo —
every concept points at a real file you can open and read.

Who this is for

- People who write code with Claude already, and want a sharper workflow
- People who don't write code, but want to understand what's happening
- Teammates who'll see PRs, branches, and "release notes" land in their inbox

If you're in group 2, you'll still get the *why* of every choice.
You don't need to understand the syntax to follow along.

What we'll cover

1. The stack: Next.js, Neon, Vercel, NextAuth, Resend — and why each
2. GitHub: what it is, what to commit, what never to commit
3. Claude Code: CLAUDE.md, agents, skills, memory
4. How I actually work with it (and one flag you should know about)
5. The review cadence — and what every review is actually *for*
6. Tricks: dev tunnels, remote work, library updates

Part 1 — The stack

Why these five pieces, and not others.

Next.js

What: A framework for building websites and web apps in one project. Pages, APIs, and database calls live side by side.

Why I use it:

- One codebase for the frontend and backend — fewer moving parts
- Files become URLs automatically — `src/app/admin/page.tsx` is `/admin`
- Deploys to Vercel with no configuration

See it in the starter: `src/app/`

Neon

What: A Postgres database that runs in the cloud and only charges when you actually use it.

Why I use it:

- Real Postgres, not a knockoff — every Postgres tutorial works
- **Branching**: you can fork the database the way you fork code. Test risky migrations on a branch first.
- Generous free tier — fine for projects in their first year of life

See it in the starter: `src/lib/db/`, `drizzle.config.ts`

Vercel

What: Where the app actually runs once we ship it. Push to GitHub → Vercel **builds** it (pulls the new code, installs dependencies, compiles the site, runs the build step) → ships the new version live in ~1–2 minutes.

Why I use it:

- Zero configuration for Next.js apps — Vercel made Next.js
- **Preview URLs**: every pull request gets its own live URL automatically. Stakeholders can poke at a change before it merges.
- Global edge network — fast for users no matter where they are

NextAuth (Auth.js v5)

What: Handles "log in" so I don't have to write it.

Why I use it:

- Don't roll your own auth. The library has handled the edge cases.
- Google sign-in out of the box — Google is what users already have an account with
- JWT-backed sessions with a clean callback for stuffing roles + features into the session

See it in the starter: `src/auth.ts`, `src/lib/auth/config.ts`

Resend

What: A service for sending transactional email — password resets, magic links, "your form was submitted," alerts.

Why I use it:

- Clean API. One function call: "send this email." Done.
- **Deliverability**: emails actually land in inboxes, not spam. Hosting your own SMTP server is a part-time job nobody wants.
- Generous free tier — 3,000 emails/month free, plenty for early-stage projects
- Good logs/UI for debugging "why didn't my email send?"

See it in the starter: `src/lib/email.ts`

Why this combo

Pain point	The piece that solves it
Slow setup to "hello world"	Next.js + Vercel
Real database without ops work	Neon
Login flows hardened by a widely used auth library	NextAuth + Google
Email that lands	Resend

All five have a free tier. You can get a working app to a real URL for **$0/month** until you have real users.

This isn't the only good stack. It's optimized for *speed to working software* and *low operational friction*. Plenty of teams happily ship on SvelteKit + Supabase, or Rails + Heroku, or whatever else fits their context. Pick your poison; the workflow patterns in the rest of this deck travel.

Part 2 — GitHub

What it is and how I use it.

What GitHub is

GitHub stores your code online and tracks every change.

Think Google Docs for code:

- Every change is recorded forever
- You can see who changed what, and when
- Multiple people can work on different parts without overwriting each other
- If you break something, you can go back to a working version

It's also where I publish open-source projects and where Vercel reads my code to build the live site.

How a change gets to "live"

Solo:

1. Claude (or I) make the change
2. **Commit** with a one-line description of what changed
3. **Push** to GitHub → Vercel builds + ships in ~2 minutes

On a team — same flow, but with a pull request:

1. Push the change to a **branch**, not main
2. Open a **pull request (PR)** — a teammate reviews the diff
3. Once approved + checks pass, merge to main → Vercel ships

For audience #3 (teammates): PRs are where you see the work. Comments live there.

GitHub — the mental model

Four words you'll see constantly. Here's the metaphor:

- `main` — the *current production history*. What's live.
- **branch** — a *sandbox* that forks off `main`. Mess around freely; nothing on main is affected.
- **PR (pull request)** — a *request for review*: "please merge my branch into main."
- **merge** — *accepting* a PR. The branch's changes become part of main → Vercel ships.

You can have many branches at once. Two teammates work on two features without ever stepping on each other's code.

GitHub Desktop

I keep **GitHub Desktop** open as a visual diff viewer, even when Claude does the committing.

- Visual list of every file that changed
- Click a file, see the before/after side by side
- Useful for the "read the diff" rule from earlier

The two patterns coexist:

- Routine changes I already reviewed → Claude commits + pushes
- Anything I want to eyeball file-by-file → I review in Desktop, *then* click Commit + Push myself

Download: **desktop.github.com**

What I commit

✅ Yes, commit:

- Source code (`.ts` , `.tsx` , `.css`)
- Configuration files (`package.json` , `next.config.ts` , `tsconfig.json`)
- Documentation (`README.md` , `CLAUDE.md` , `docs/`)
- The Claude workspace (`.claude/agents/` , `.claude/skills/`)
- **`.env.example`** — the *shape* of the secrets file, with placeholders

What I never commit

❌ Never commit:

- `.env` / `.env.local` — these have real secrets in them
- API keys, passwords, database URLs with credentials
- `node_modules/` — generated, huge
- Build output (`.next/`, `dist/`) — generated
- Personal IDE settings

How the starter prevents accidents: a `.gitignore` file lists patterns to skip. `.env.local` is on that list.

Claude knows these rules — but **you should also know them**, because mistakes here are how secrets leak.

.env.example vs .env.local

File	Goes in git?	What's in it
`.env.example`	✅ Yes	`RESEND_API_KEY=` (just the key name, no value)
`.env.local`	❌ Never	`RESEND_API_KEY=re_actualSecretKey…`

When you fork the starter:

1. Copy `.env.example` → `.env.local`
2. Fill in real values
3. Your `.env.local` stays on your machine, period.

Part 3 — Claude Code

The tools that make Claude useful for real software work.

CLAUDE.md

The "house rules" file in every project.

Every time I start a session with Claude in a repo, it reads `CLAUDE.md` first. It tells Claude:

- What the project is
- How the code is organized
- What patterns we use here (vs other places)
- What *not* to do (past mistakes, project-specific gotchas)

Open it now: `CLAUDE.md` in the starter — the whole workflow lives there.

CLAUDE.md — what's in the starter's

- Project overview & stack
- **"How this user works"** section (the flag we'll talk about later)
- The 6-phase pipeline (every feature goes through these phases)
- The review cadence (test-coverage, retro, code, docs, security, agents, dependencies)
- Document naming conventions
- Common commands
- Key invariants

You'd read it once when starting, then forget it exists — Claude does the work of remembering.

Agents

Specialized Claudes that focus on one job.

The starter ships with 9 agents in `.claude/agents/`:

- **analyst** — "what is the user trying to do?"
- **architect** — "where does this code belong?"
- **tech-lead** — "what's the design?"
- **api-developer** — server-side & route handlers
- **ux-developer** — pages, components, accessibility
- **full-stack-developer** — features that cross both
- **database-admin** — schema + migrations
- **deployment-engineer** — Vercel + envs + secrets
- **qa** — tests + coverage

Each is just a markdown file describing the role and its responsibilities.

Agents — how they get used

Two ways:

1. **Automatic** — Claude recognizes "oh, this is an architectural decision" and delegates to the architect agent on its own.
2. **Explicit** — I say "have the architect review this" and it spawns one.

When Claude fans out *multiple* agents at once (explicitly or because the task has independent parts), they run **in parallel**:

- Faster (multiple Claudes thinking at once)
- Cleaner (the main conversation doesn't fill up with research)

A single delegated agent is still just one Claude — "parallel" only kicks in when there's more than one to run.

Open `.claude/agents/architect.md` to see how an agent is defined.

The pipeline — five beats

Every non-trivial feature moves through these five beats. The agents map onto them:

1. **Understand** — what's the user actually trying to do? *(analyst)*
2. **Design** — where does this code live and what's the shape? *(architect + tech-lead)*
3. **Implement** — write the code. *(developers)*
4. **Verify** — does it work and does it not break anything else? *(qa)*
5. **Compare against intent** — does the shipped thing match what we set out to build? *(analyst again)*

Full gate criteria for each beat live in `CLAUDE.md` — the deck shows the rhythm, the file shows the discipline.

Skills

Reusable mini-procedures invoked with `/` commands.

The starter ships with 5 skills in `.claude/skills/`:

- **`/new-feature`** — walk a new feature through the 6 phases
- **`/add-permission`** — wire a new permission into the catalog
- **`/release-notes`** — generate the release-notes entry for what shipped
- **`/pre-push`** — run typecheck + build + drizzle check before pushing
- **`/neon-postgres`** — Neon-specific patterns (branches, migrations)

You type `/release-notes` in Claude's input, it follows the recipe in `SKILL.md`.

Memory

There are two kinds of memory in Claude Code:

- **CLAUDE.md** — memory **you** write. Project rules, conventions, gotchas. Lives in the repo. Covered on slide 19.
- **Auto memory** — memory **Claude** writes for itself. Things it learned about you or the work, saved to `~/.claude/projects/.../memory/`. Loads automatically next session.

This slide is about the second kind. Four types of auto memory:

Type	Example
user	"Uses Next.js + Neon as default stack"
feedback	"Claude runs frequent commands, not me"
project	"Auth rewrite is for compliance, not tech-debt"
reference	"Bugs tracked in Linear project INGEST"

Auto memory requires Claude Code v2.1.59 or later.

Memory — tips

- If you tell Claude something important *about how you work*, ask it to **remember**. Otherwise it forgets at the end of the session.
- Memory should be **specific** and **non-obvious**. "Uses Python" is bad. "Prefers requests over httpx for this team's services because of an existing wrapper" is good.
- You can ask Claude to **forget** something that's wrong or stale.
- Memory is per-user, per-project (mostly). It doesn't leak between repos.

Save-state before you restart

The trick I just used in this session.

When you need to **restart Claude** — context filling up, debug session went sideways, you're rebooting, you're switching machines — tell Claude:

> "Remember what we're doing. I'm going to restart Claude."

Claude writes the active bug, the half-finished work, the file paths, the next step, and any in-flight uncommitted changes to a project memory file.

The **next session reads `MEMORY.md` on start** and picks up exactly where the previous one left off — same diagnosis, same plan, no "wait, what were we doing?"

Pair this with frequent commits and you can restart Claude without losing your place.

Part 4 — How I actually work with Claude

The pragmatic stuff.

`--dangerously-skip-permissions`

I run Claude with this flag.

What it does: Claude doesn't ask permission before running commands. No "may I run `npm install`?" pop-up.

Why: The pop-ups slow you down to a crawl when you trust Claude to know what it's doing.

The tradeoff: Real responsibility. Claude can:

- Delete files
- Rewrite history
- Push to remotes
- Run database migrations

It's a calculated bet — Claude is good, I check the diff, and `git` is a safety net.

Plan mode is the brake (next slide). **Not for everyone** — new users should not start

I have Claude run commands, not me

Paired with the flag above:

- Starting the dev server → Claude does it (in the background)
- Running tests → Claude does it
- Building, typechecking, migrating → Claude does it

It would be silly to opt out of permission prompts and *then* have Claude tell me "run `npm run dev`."

This is **baked into the starter's CLAUDE.md** under "How This User Works." Future Claudes that read it will do the same.

Effective collaboration — hints

- **Be specific.** "Fix the login" is bad. "The `/signin` page errors when `callbackUrl` is empty — see browser console" is good.
- **Point at files by path.** `src/auth.ts:42` is gold.
- **Paste error messages verbatim.** Don't summarize. The exact text matters.
- **Use plan mode** (`Shift+Tab` twice) for risky changes — Claude shows the plan before doing anything.
- **Read the diff.** Always. Especially with `--dangerously-skip-permissions`.

Effective collaboration — anti-patterns

- ❌ "Make it better." → No signal, low-value output.
- ❌ Asking Claude to keep guessing when it's clearly off-track. Stop, give context, restart.
- ❌ Long monologues describing the codebase. Just say "read `src/auth.ts`" — it can.
- ❌ Burying the question at the end of a long message. Put the ask at the top.

Plan mode — the brake

If `--dangerously-skip-permissions` is the accelerator, **plan mode is the brake.**

Toggle: press `Shift+Tab` twice in the Claude prompt.

In plan mode:

- Claude **cannot** edit files, run commands, or call destructive tools
- It can still read, search, and think
- It produces a written plan you approve before *anything* runs

When I reach for it:

- Database migrations that will touch real data
- Refactors that span more than ~5 files
- Anything I can't easily reverse with `git reset`
- When I'm not sure Claude has the full picture yet

What does this cost?

Piece	Free tier	Paid kicks in when
Vercel	Hobby plan: free for personal use	You need team features or commercial use
Neon	0.5 GB storage + 191 compute-hours/mo free	You scale past a hobby app
Resend	3,000 emails/month free (verify current)	You send more email than that
NextAuth	Free (it's a library)	Never
GitHub	Free for public + small private	You need enterprise features
Claude Code	See pricing → claude.com/pricing	Heavy daily use

Realistic: you can run a real, useful app for **$0/month** until it has actual users. Claude Code itself is the meaningful line item — and even there, you only pay for what you use.

When Claude is wrong

Real talk: Claude will sometimes do the wrong thing. The safety net is **git**.

```
# Undo all uncommitted changes in a file
git checkout -- src/some-file.ts

# Undo all uncommitted changes in the whole working tree
git checkout -- .

# Roll back to the last commit (keeps changes staged for review)
git reset --soft HEAD~1

# Nuke the last commit entirely (use carefully)
git reset --hard HEAD~1
```

The big promise: GitHub keeps every version of every file forever. Until you `git push --force` over your own history (rare, intentional), nothing is gone.

If you commit small and commit often, the "undo" is always cheap.

Part 5 — The review cadence

Why we review on a schedule.

The cadence

Review	Cadence	Owner
Test coverage	7 days	qa
Retrospective	7 days	all agents → tech-lead synthesizes
Code	30 days	architect
Docs	30 days	tech-lead
Security	30 days	api-developer + database-admin
Agent instructions	30 days	tech-lead
Dependencies	30 days	deployment-engineer

Each one has a *reason* — see next slide.

Why each review exists

- **Test coverage (7d)** — Code outpaces tests. Catches the drift before it's too painful to fix.
- **Retrospective (7d)** — Improves CLAUDE.md and the agents themselves. Compound learning.
- **Code review (30d)** — Quality drift, dead code, places where patterns diverged.
- **Docs review (30d)** — Code changed, docs didn't. Reality check.
- **Security (30d)** — New attack surface ships every week. Look at it on a cadence.
- **Agent instructions (30d)** — Agents drift the same way docs do. Prune what's wrong.
- **Dependencies (30d)** — Old libraries have CVEs. **Keep them current as a security practice.**

A note on dependencies

Keeping libraries up to date is a security responsibility, not a chore.

- Vulnerabilities are reported against old versions.
- Supply-chain attacks target abandoned packages.
- The longer you wait, the bigger the upgrade gap, the more breaks.

`npm audit` will tell you what's vulnerable. `npm outdated` will tell you what's stale.

The deployment-engineer agent owns this every 30 days.

Part 6 — Tricks

Things that make life easier.

Remote-enabled Claude Code

You can run Claude Code on a **remote machine** and drive it from your laptop — SSH in, run `claude`, work as if it were local.

When it helps:

- The repo is huge and your laptop is slow
- You're working on a server you'd never copy locally
- You want the same Claude session from multiple machines

Setup: if you already SSH into a dev box, you're 90% there — install Claude Code on the remote, then SSH in and run `claude` like you would locally. Tmux/screen for persistent sessions.

Cloudflare tunnel — let outsiders test your laptop

Sometimes you need a stakeholder to **click a real URL** of work that hasn't shipped yet:

- The feature is on your laptop
- You don't want to deploy a preview every 3 minutes
- The stakeholder is on a different network

`cloudflared tunnel` gives you a public HTTPS URL that points at your laptop's dev server.

```
cloudflared tunnel --url http://localhost:3000
```

Out pops a URL. You send it. They click. It works.

Cloudflare tunnel — when to use it

- **UAT (user acceptance testing)** before a feature is even on a preview URL
- **Pairing** with someone in a different city
- **Demos** of half-finished work to a non-technical stakeholder
- **Mobile testing** without setting up your network

Stop the tunnel when you're done — the URL dies with it.

MCP — Claude talks to your tools

MCP (Model Context Protocol) lets Claude Code call out to **other systems** while you're working: Linear, Notion, Slack, Google Drive, your CRM, your monitoring dashboards.

You configure an "MCP server" for each tool. Claude can then:

- Read Linear tickets to pull context into a feature
- Update a Notion page when work ships
- Search Drive for a spec doc instead of you copy-pasting it
- Post a release-notes summary to Slack

Why it matters for teams: the work doesn't happen only in the repo. MCP brings the rest of your workflow into the same conversation.

Configure under `claude mcp` or in your settings — start with one server, expand as it pays off.

Wrapping up

What you got from this starter and this deck:

- A real fork-and-go template — login, admin, roles, 2FA, flags, audit log
- A working `.claude/` setup — agents, skills, CLAUDE.md, settings
- An SDLC playbook — phases, cadence, decisions, work-log, release-notes
- The reasoning behind every piece, in plain language

Where to go next

1. **Fork the starter** — `gh repo clone chenson42/claudecode my-new-app`
2. **Set up** `.env.local` from `.env.example`
3. **Run** `npm run db:push && npm run db:seed` to spin up your database
4. **Sign in once** — your email lands in the seeded admin role
5. **Build your thing** — and let Claude do the typing

Questions?

Find me at chenson42@gmail.com.

Source: **github.com/chenson42/claudecode**
Slides source: `deck/slides.md`
Rendered PDF: `deck/slides.pdf` (downloadable from GitHub)